

1 Motivation and Industry Applications

2 Transformer Architecture

3 Encoder Architecture

4 Decoder Architecture

5 Vision Transformer

6 Transformer Variants

7 Using Pre-trained Models

3 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

THE RNN BOTTLENECK

• Recurrent Neural Networks (RNNs):

- ▶ Process sequences one token at a time: $h_t = f(h_{t-1}, x_t)$
- ▶ Hidden state carries information forward
- ▶ Used with attention

• Three Critical Problems:

- ▶ **Sequential Processing:** Cannot parallelize
 - ★ Must compute h_1 before h_2 before $h_3...$
 - ★ Training is slow (hours → days → weeks)
- ▶ **Vanishing Gradients:** Long-range dependencies fail
 - ★ Information from early tokens fades
 - ★ Even with LSTM/GRU, struggles beyond 100-200 tokens
- ▶ **Fixed Memory:** Hidden state is bottleneck
 - ★ All information compressed into fixed-size vector
 - ★ Limited capacity as sequences grow

• Question: Can we build sequence models without recurrence?

5 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

• What We Learned:

- ▶ Attention allows models to focus on relevant parts of input
- ▶ Query-Key-Value framework: Attention as information retrieval
- ▶ Multiple attention types:
 - ★ Self-attention: Q, K, V from same sequence
 - ★ Multi-head: Parallel attention with different projections
 - ★ Cross-attention: Q from one sequence, K,V from another
 - ★ Masked: Prevent attending to future tokens

• What's Missing?

- ▶ How to organize these attention mechanisms into a complete architecture?
- ▶ How to handle sequences without recurrence?
- ▶ How to scale to billions of parameters?

• This Module:

- ▶ Transformer architecture: Attention as the only mechanism
- ▶ Three variants: Encoder-only, Decoder-only, Encoder-decoder

4 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

THE TRANSFORMER SOLUTION (2017)

• Key Insight: "Attention is All You Need"

- ▶ Replace recurrence entirely with self-attention
- ▶ Each token directly attends to all other tokens
- ▶ No sequential dependency → full parallelization

• How Transformers Solve RNN Problems:

Problem	RNN	Transformer
Parallelization	Sequential	Parallel
Long-range deps	Vanishing gradients	Direct connections
Memory	Fixed hidden state	Attention to all tokens

• The Impact:

- ▶ 2017: Machine translation breakthrough
- ▶ 2018: BERT revolutionizes NLP (encoder-only)
- ▶ 2018-2020: GPT series (decoder-only)
- ▶ 2020-present: Foundation of all large language models

• Result: Transformers became universal architecture for sequences

6 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FROM TRANSFORMERS TO MODERN AI



Transformers Enable Scaling:

- Parallel processing → train on massive datasets
- Direct connections → learn complex patterns
- Flexible architecture → adapt to any domain

This Scaling Led To:

1 Large Language Models (LLMs)

- 100B+ parameters: GPT-4, Claude, Gemini
- Trained on trillions of tokens
- General intelligence: reasoning, coding, writing

2 Small Language Models (SLMs)

- 1B-7B parameters: Phi-3, Gemma, Mistral-7B
- On-device deployment: phones, laptops
- Privacy-preserving, low-latency

3 Specialized Models

- Vision: ViT, CLIP (images without convolutions)
- Speech: Whisper, Wav2Vec2 (audio understanding)
- Multimodal: GPT-4V, Gemini (text + images + audio)

7 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

REAL-WORLD APPLICATIONS



1 Conversational AI

- ChatGPT, Claude, Gemini: Natural dialogue with context
- Customer support automation: 24/7 intelligent responses
- Virtual assistants: Multi-turn conversations with memory

2 Code Generation & Software Development

- GitHub Copilot: Real-time code completion
- CodeLlama, StarCoder: Function generation from descriptions

3 Content Creation

- Writing assistance: Articles, emails, reports
- Summarization: Long documents → key points
- Translation: 100+ languages with context

4 Scientific Research

- AlphaFold 3: Protein structure prediction
- Drug discovery: Molecular design and screening

5 Next: How do these models differ? Three transformer variants

8 / 1

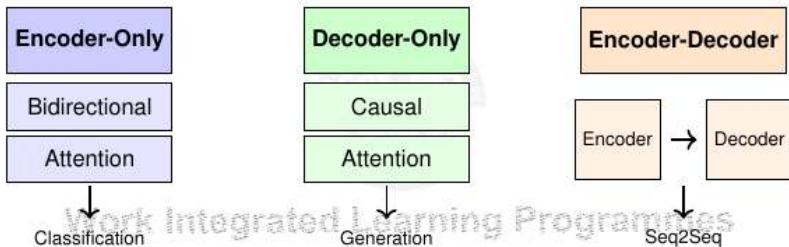
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

THREE TRANSFORMER ARCHITECTURES



Why Three Variants?

- Different tasks need different information flow
- Trade-off between understanding vs generation
- Computational efficiency for specific use cases



Selection Criteria:

- Need to classify or extract? → Encoder-only
- Need to generate text? → Decoder-only
- Input and output very different? → Encoder-decoder

9 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

INDUSTRY MODELS COMPARISON



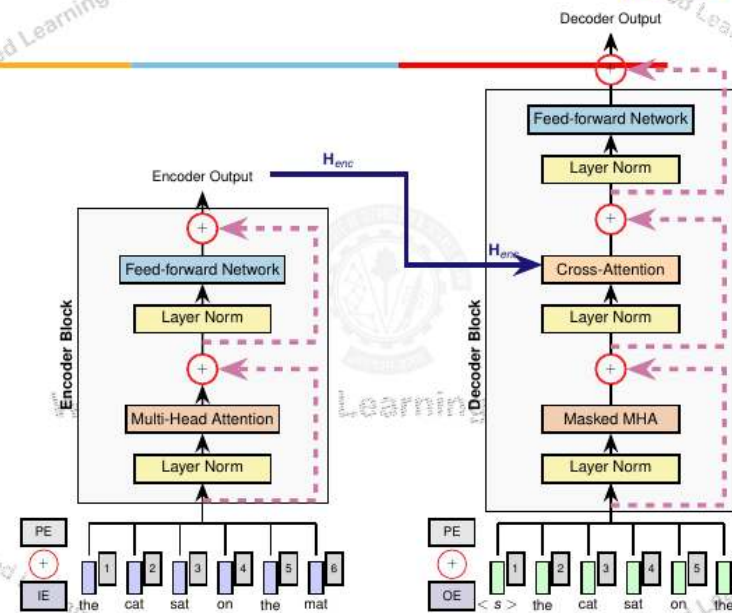
Model	Arch	Parameters	Strengths	Access
GPT-4	Decoder	1.8T	Reasoning, code	API (OpenAI)
Claude 4	Decoder	200B	Safety, long context	API (Anthropic)
Gemini 3	Decoder	175B	Multimodal	API (Google)
LLaMA 3	Decoder	8B-405B	Open, efficient	Open (Meta)
Mistral	Decoder	7B-123B	Fast inference	Open (Mistral AI)
BERT	Encoder	110M-340M	Understanding, NER	Open (Google)
T5	Enc-Dec	60M-11B	Translation, summary	Open (Google)

Key Observations:

- Decoder-only dominates generative tasks
- Open models (LLaMA, Mistral) competitive with proprietary
- Parameter count vs performance: Not always linear

10 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



WHAT DOES THE ENCODER DO?

• Purpose: Transform input sequence into rich contextual representations

- ▶ Input: Raw tokens (words, subwords, patches)
- ▶ Output: Context-aware embeddings for each input position

• Key Capability: Bidirectional Context

- ▶ Each token attends to **all other tokens** (no restrictions)
- ▶ Captures relationships in both directions
- ▶ Example: "bank" in "river bank" vs "bank account"

• How it Works:

- ▶ Multi-Head Attention: Tokens "communicate" with each other
- ▶ Feed-Forward Network: Each token "thinks" independently
- ▶ Stacked N times: Builds increasingly abstract representations

• Output:

- ▶ $H_{enc} \in \mathbb{R}^{n \times d_{model}}$ where n is sequence length
- ▶ Each row is a contextualized representation of one input token
- ▶ Used by decoder (encoder-decoder) or task head (encoder-only)

WHAT DOES THE DECODER DO?

• Purpose: Generate output sequence one token at a time

- ▶ Input: Previously generated tokens (autoregressive)
- ▶ Output: Probability distribution over next token

• Key Capability: Causal Generation

- ▶ Token at position t can only attend to positions $\leq t$
- ▶ Prevents "looking ahead" to future tokens
- ▶ Ensures model learns to predict sequentially

• How it Works:

- ▶ Masked Self-Attention: Tokens attend only to past
- ▶ Cross-Attention: Attends to encoder output (encoder-decoder only)
- ▶ Feed-Forward Network: Process information independently
- ▶ Stacked N times: Refines generation at each layer

• Output:

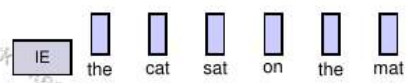
- ▶ $D_{out} \in \mathbb{R}^{m \times d_{model}}$ where m is target length
- ▶ Select next token (greedy/sampling/beam search)

TABLE OF CONTENTS

- 1 Motivation and Industry Applications
- 2 Transformer Architecture
- 3 Encoder Architecture
- 4 Decoder Architecture
- 5 Vision Transformer
- 6 Transformer Variants
- 7 Using Pre-trained Models

TRANSFORMER ENCODER: INPUT EMBEDDING

- Input Embedding:
 - Convert tokens to dense vectors:
 $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$
 - Learned embedding matrix:
 $\mathbf{E} \in \mathbb{R}^{|V| \times d_{\text{model}}}$
 - Typical $d_{\text{model}} = 512$ (base) or 1024 (large)

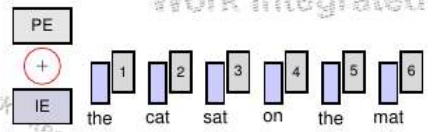


TRANSFORMER ENCODER: POSITION EMBEDDING

- Positional Encoding:
 - Problem: Attention has no notion of position
 - Solution: Add position information to embeddings

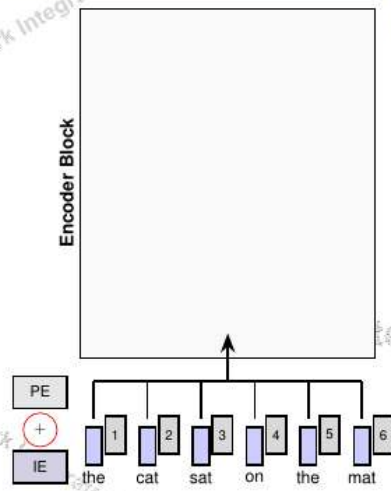
$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \tag{1}$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \tag{2}$$



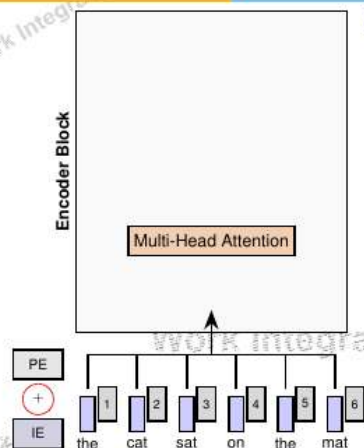
TRANSFORMER ENCODER: ENCODER INPUT

- Final Encoder Input: = Input Embedding concatenate Position Embedding
- $\mathbf{X} = \text{Embedding}(\text{tokens}) + \text{PE}(\text{positions})$ (3)





TRANSFORMER ENCODER: MHA



Multi-Head Attention:

- Allows each embedding to attend to all other embeddings
- h parallel attention heads capture different relationships
- For each head $h = 1, \dots, H$:

$$\mathbf{Q}_h = \mathbf{Z}\mathbf{W}_h^O, \quad \mathbf{K}_h = \mathbf{Z}\mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{Z}\mathbf{W}_h^V \quad (4)$$

$$\text{head}_h = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^T}{\sqrt{d_k}}\right) \mathbf{V}_h \quad (5)$$

$$\text{MHA}(\mathbf{Z}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O \quad (6)$$

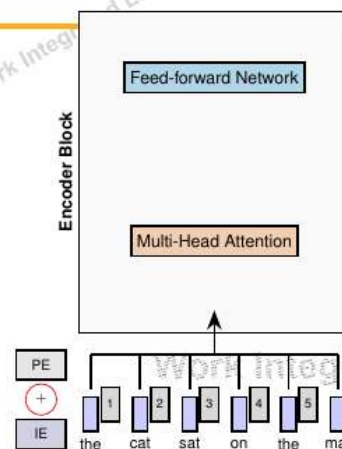
- Typical: $H = 12$ heads, $d_k = d_v = D/H = 64$

19 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TRANSFORMER ENCODER: FFN



Feed-Forward Network (Position-wise):

- FFN "Thinks" **independently** and processes the information
- Two-layer FFN with expansion
 - Expand:** $D \rightarrow 4D$; $\mathbf{W}_1 \in \mathbb{R}^{D \times 4D}$ parameters
 - Non-linearity:** RELU activation
 - Applied **independently** to each position
 - Contract:** $4D \rightarrow D$; $\mathbf{W}_2 \in \mathbb{R}^{4D \times D}$ parameters

Why Position-wise FFN?

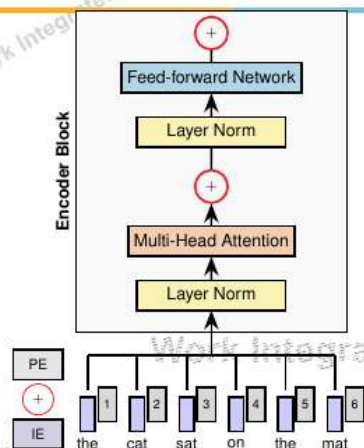
- Same weights applied at each position independently
- Adds non-linearity after attention
- Provides model capacity for complex transformations

20 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TRANSFORMER ENCODER: LAYER NORM



Problem: Training Deep Networks is Hard

- Stacking many layers $\rightarrow$ vanishing gradients
- Information loss
- Optimization becomes unstable

Solution 1: Layer Normalization

- Normalize across feature dimension
- Stabilizes training across features (not batch)
- Learnable parameters: $\gamma, \beta \in \mathbb{R}^{d_{\text{model}}}$
- Same procedure at train/test time

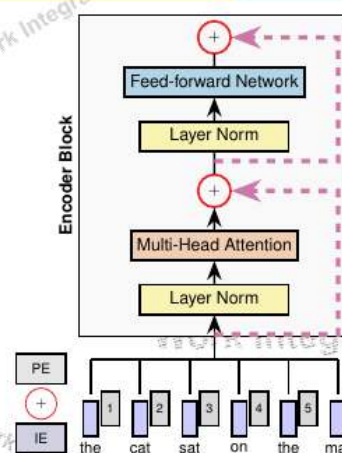
$$\text{LN}(\mathbf{x}) = \gamma \frac{\mathbf{x} - \mathbb{E}[\mathbf{x}]}{\sqrt{\text{Var}[\mathbf{x}] + \epsilon}} + \beta \quad (7)$$

21 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TRANSFORMER ENCODER: RESIDUAL CONNECTIONS



- Problem: Training Deep Networks is Hard
- Solution 2: Residual Connections (Skip Connections)

- Help gradient flow in deep networks (from ResNet)
- Add input directly to output:

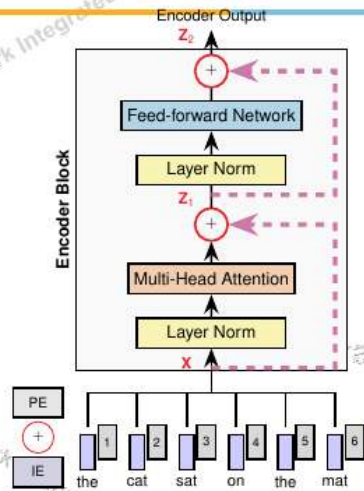
$$\text{Out} = \mathbf{x} + \text{Sublayer}(\mathbf{x}) \quad (8)$$

- Creates "gradient highways" $\rightarrow$ easier optimization
- Allows training very deep models (100+ layers)

22 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER ENCODER: ENCODER OUTPUT



- Complete Encoder Layer:

$$\mathbf{Z}_1 = \mathbf{X} + \text{MHA}(\text{LayerNorm}(\mathbf{X})) \quad (9)$$

$$\mathbf{Z}_2 = \mathbf{Z}_1 + \text{FFN}(\text{LayerNorm}(\mathbf{Z}_1)) \quad (10)$$

23 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER ENCODER: PARAMETERS

Each token gets embedded into a d_{model} -dimensional vector, and we can process sequences up to n_{max} tokens long.

- $|V|$: Vocabulary size (30K-50K tokens)
- d_{model} : Embedding vector length/dimension (512 (base) or 1024 (large))
- n_{max} : Maximum sequence length (512-2048 tokens)

Input Embedding: $|V| \times d_{\text{model}}$

Positional Encoding: $n_{\text{max}} \times d_{\text{model}}$ (learnable)

Layer Norm (2 layers): $2 \times 2 \times d_{\text{model}} = 4d_{\text{model}}$

Multi-Head Attention: $4 \times d_{\text{model}}^2$ (for W^Q, W^K, W^V, W^O)

Feed-Forward Network: $2 \times d_{\text{model}} \times d_{\text{ff}} = 8d_{\text{model}}^2$ where $d_{\text{ff}} = 4d_{\text{model}}$

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$$

Total per layer: $4d_{\text{model}} + 4d_{\text{model}}^2 + 8d_{\text{model}}^2 \approx 12d_{\text{model}}^2$

24 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ENCODER STACK

- Stack of N Identical Layers:

- Original Transformer: $N = 6$ layers
- BERT-base: $N = 12$ layers
- BERT-large: $N = 24$ layers
- GPT-3: $N = 96$ layers (decoder, but same principle)

- Stacking Process:

$$\mathbf{H}^{(\ell)} = \text{EncoderLayer}^{(\ell)}(\mathbf{H}^{(\ell-1)}), \quad \ell = 1, \dots, N \quad (11)$$

where $\mathbf{H}^{(0)} = \text{InputEmbedding} + \text{PositionalEncoding}$

- Encoder Output:

- Final encoder output: $\mathbf{H}^{(N)} \in \mathbb{R}^{n \times d_{\text{model}}}$
- Rich contextual representation for each input token
- Each position has attended to all other positions N times
- Passed to decoder via cross-attention (in encoder-decoder models)

25 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

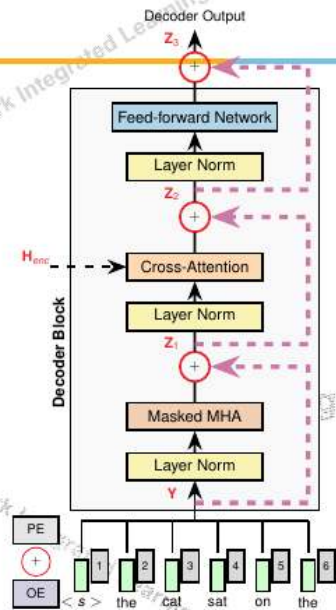
TABLE OF CONTENTS

- 1 Motivation and Industry Applications
- 2 Transformer Architecture
- 3 Encoder Architecture
- 4 Decoder Architecture
- 5 Vision Transformer
- 6 Transformer Variants
- 7 Using Pre-trained Models

26 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER DECODER: DECODER OUTPUT



Complete Decoder Layer:

$$Z_1 = Y + \text{MaskedMHA}(\text{LayerNorm}(Y)) \quad (12)$$

$$Z_2 = Z_1 + \text{CrossAttn}(\text{LayerNorm}(Z_1), H_{enc}) \quad (13)$$

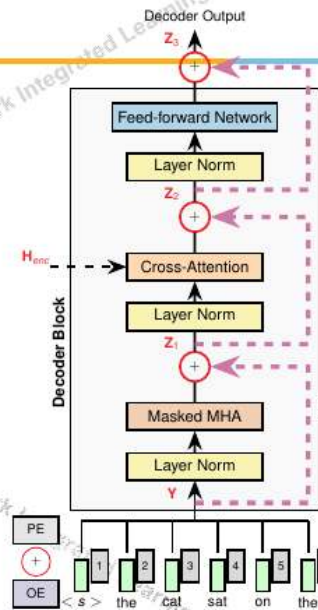
$$Z_3 = Z_2 + \text{FFN}(\text{LayerNorm}(Z_2)) \quad (14)$$

where H_{enc} is encoder output

27 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER DECODER: MASKED MULTI-HEAD ATTENTION



Self-attention with causal masking

- Token t attends only to positions $\leq t$
- Prevents "looking ahead" to future tokens

Why? Match training and inference

- At inference: Generate one token at a time
- Training must simulate this autoregressive process

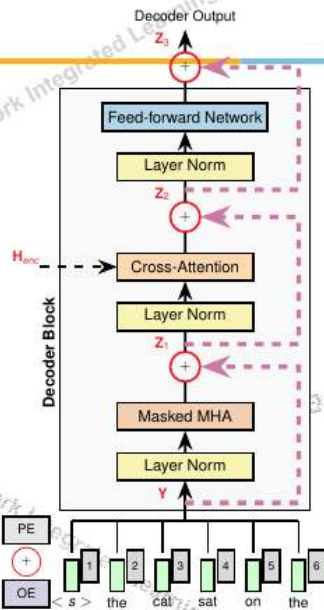
$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \quad (15)$$

$$\text{MaskedAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \text{mask}\right) \mathbf{V} \quad (16)$$

28 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER DECODER: CROSS-ATTENTION



Cross-Attention (Encoder-Decoder Attention):

- Decoder attends to encoder output
 - Query (Q): From decoder layer
 - Key (K), Value (V): From encoder output H_{enc}
 - No masking (can attend to all source tokens)
- Why needed:** Connect source and target
 - Allows decoder to focus on relevant source positions

$$\text{CrossAttn}(\mathbf{Q}_{dec}, \mathbf{K}_{enc}, \mathbf{V}_{enc}) = \text{softmax}\left(\frac{\mathbf{Q}_{dec}\mathbf{K}_{enc}^T}{\sqrt{d_k}}\right) \mathbf{V}_{enc}$$

$$\mathbf{Q}_{dec} = \text{LN}(Z_1)W^Q$$

$$\mathbf{K}_{enc} = H_{enc}W^K$$

$$\mathbf{V}_{enc} = H_{enc}W^V$$

29 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRANSFORMER DECODER: PARAMETERS

Each output token gets embedded into a d_{model} -dimensional vector.

- Same vocabulary and embedding dimensions as encoder
- $|V|$, d_{model} , n_{max} same as encoder

Output Embedding: $|V| \times d_{\text{model}}$

Positional Encoding: $n_{\text{max}} \times d_{\text{model}}$ (learnable)

Layer Norm (3 layers): $3 \times 2 \times d_{\text{model}} = 6d_{\text{model}}$

Masked Multi-Head Attention: $4 \times d_{\text{model}}^2$ (for W^Q, W^K, W^V, W^O)

Cross-Attention: $4 \times d_{\text{model}}^2$ (for W^Q, W^K, W^V, W^O)

Feed-Forward Network: $2 \times d_{\text{model}} \times d_{\text{ff}} = 8d_{\text{model}}^2$ where $d_{\text{ff}} = 4d_{\text{model}}$

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$$

Total per layer: $6d_{\text{model}} + 8d_{\text{model}}^2 + 8d_{\text{model}}^2 \approx 16d_{\text{model}}^2$

Note: Decoder has 33% more parameters than encoder (16 vs 12 d_{model}^2)

30 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DECODER STACK

Stack of N Identical Decoder Layers:

- Original Transformer: $N = 6$ layers
- Each layer has **3 sublayers** (vs 2 in encoder):
 - Masked multi-head self-attention
 - Cross-attention to encoder output
 - Position-wise feed-forward network

Stacking Process:

$$\mathbf{D}^{(\ell)} = \text{DecoderLayer}^{(\ell)}(\mathbf{D}^{(\ell-1)}, \mathbf{H}_{enc}), \quad \ell = 1, \dots, N \quad (17)$$

where $\mathbf{D}^{(0)} = \text{OutputEmbedding} + \text{PositionalEncoding}$

Decoder Output:

- Final decoder output: $\mathbf{D}^{(N)} \in \mathbb{R}^{m \times d_{\text{model}}}$ where m is target length
- Each position has attended to previous positions N times (masked)
- Each position has attended to all encoder positions N times (cross-attention)

31 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

OUTPUT LAYER: LINEAR + SOFTMAX

Purpose: Convert decoder output to vocabulary probabilities

Linear Projection:

$$\text{logits} = \mathbf{D}^{(N)} \mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}} \quad (18)$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$, $\mathbf{b}_{\text{out}} \in \mathbb{R}^{|V|}$

Softmax: Output: Probability distribution over vocabulary

$$P(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(\text{logits}_t) \quad (19)$$

Complete Flow:

$$\mathbb{R}^{d_{\text{model}}} \xrightarrow{\text{Linear}} \mathbb{R}^{|V|} \xrightarrow{\text{Softmax}} \text{Probabilities} \xrightarrow{\text{Token Selection}} y_t$$

Token selection done via greedy, sampling, or beam search

32 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

- 1 Motivation and Industry Applications
- 2 Transformer Architecture
- 3 Encoder Architecture
- 4 Decoder Architecture
- 5 Vision Transformer
- 6 Transformer Variants
- 7 Using Pre-trained Models

33 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FROM NLP TO VISION - WHY ViTs WORK

Traditional Approach: CNNs

- Convolutional layers extract local features
- Pooling layers for spatial hierarchy
- Inductive bias: Translation equivariance, locality

The ViT (Vision Transformer) Innovation (2020):

- Treat image as sequence of patches
- Apply pure transformer (no convolutions)
- Learn visual relationships through self-attention

Key Insight:

- CNNs: Hard-coded locality through convolution kernel
- ViTs: Learn relationships between all patches
- Trade-off: Need more data, but more flexible

When ViTs Excel:

- Large-scale pretraining (ImageNet-21K, JFT-300M)
- Transfer learning to downstream tasks

34 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

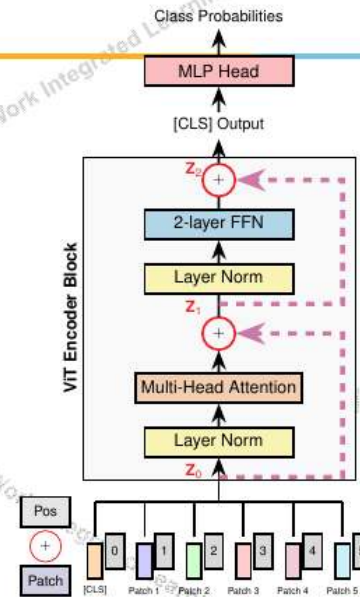
WHAT DOES VISION TRANSFORMER (ViT) DO?

- Purpose: Image classification using pure transformer (no convolutions)
- Architecture: Encoder-only (no decoder needed for classification)
- Input → Output:
- Key Innovation: Treat image as sequence of patches
 - ▶ Divide image into fixed-size patches (16×16)
 - ▶ Flatten each patch → Linear projection
 - ▶ Add positional encoding
 - ▶ Process through transformer encoder
- How Classification Works:
 - ▶ Prepend [CLS] token to patch sequence
 - ▶ Transformer encoder processes all patches
 - ▶ [CLS] output → MLP head → Class probabilities
- Why It Works: Self-attention captures spatial relationships globally

35 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

VISION TRANSFORMER (ViT): ARCHITECTURE



- Architecture: Encoder-only (no decoder)
 - ▶ Input: Image → Output: Class probabilities
 - ▶ Uses only transformer encoder blocks
 - ▶ No decoder needed for classification task
- Classification [CLS] Token:
 - ▶ Learnable special token prepended to patch sequence
 - ▶ Aggregates global image information via self-attention
 - ▶ Final [CLS] representation used for classification
- Processing Flow:

Image → Patches → Encoder → [CLS] → Class

36 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

PATCH EMBEDDING & [CLS] TOKEN

- Image to Patches:
 - ▶ Input image: $I \in \mathbb{R}^{H \times W \times C}$ (e.g., $224 \times 224 \times 3$)
 - ▶ Divide into N patches of size $P \times P$ (typically $P = 16$)
 - ▶ Number of patches: $N = \frac{H \times W}{P^2} = \frac{224 \times 224}{16 \times 16} = 196$

- Linear Projection:

$$\mathbf{z}_i = \text{Flatten}(\text{patch}_i) W_p, \quad W_p \in \mathbb{R}^{(P^2 \cdot C) \times d} \quad (20)$$

- ▶ Flatten each patch: $16 \times 16 \times 3 = 768$ pixels
- ▶ Project to embedding dimension d (typically 768)
- ▶ Result: $\mathbf{Z} = [\mathbf{z}_1; \mathbf{z}_2; \dots; \mathbf{z}_N] \in \mathbb{R}^{N \times d}$

- Prepend [CLS] Token:

- ▶ Learnable embedding $\mathbf{z}_{[\text{CLS}]}$ prepended to sequence
- ▶ Aggregates global image information via self-attention
- ▶ Final sequence: $[\mathbf{z}_{[\text{CLS}]}; \mathbf{z}_1; \mathbf{z}_2; \dots; \mathbf{z}_N] \in \mathbb{R}^{(N+1) \times d}$

37 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

POSITIONAL ENCODING

- Problem: Self-attention is permutation invariant
 - ▶ Model cannot distinguish patch positions without explicit encoding
 - ▶ Order of patches matters for understanding image structure
- Solution: Learnable Positional Embeddings
 - ▶ $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times d}$ where $N + 1 = 197$ positions
 - ▶ One embedding for each position: [CLS] (position 0) + 196 patches
 - ▶ Learned during training (not fixed like sinusoidal in transformers)
 - ▶ 1D embeddings sufficient; no 2D structure hardcoded
- Adding Position Information:

$$\mathbf{z}_0^{(0)} = [\mathbf{z}_{[\text{CLS}]}; \mathbf{z}_1; \mathbf{z}_2; \dots; \mathbf{z}_N] + \mathbf{E}_{\text{pos}} \quad (21)$$

- ▶ Element-wise addition of position embeddings
- ▶ Input to first encoder layer: $\mathbf{z}_0^{(0)} \in \mathbb{R}^{(N+1) \times d}$

38 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ViT ENCODER: PROCESSING



- Encoder Layer Operations:
 - Same as Transformer encoder:
Layer Norm → MHA → Add → Layer Norm → FFN → Add
 - Stack of L identical layers (ViT-Base: $L = 12$)
- Layer ℓ aligns: $\mathbf{z}_0^{(\ell-1)} \in \mathbb{R}^{(N+1) \times d}$ is input to layer ℓ

$$\mathbf{z}_1^{(\ell)} = \mathbf{z}_0^{(\ell-1)} + \text{MHA}(\text{LayerNorm}(\mathbf{z}_0^{(\ell-1)})) \tag{22}$$

$$\mathbf{z}_2^{(\ell)} = \mathbf{z}_1^{(\ell)} + \text{FFN}(\text{LayerNorm}(\mathbf{z}_1^{(\ell)})) \tag{23}$$

- Multi-Head Self-Attention:
 - Each patch (including [CLS]) attends to all other patches
 - Captures global spatial relationships
 - ViT-Base: $H = 12$ heads, $d_k = d/H = 64$
- Feed-Forward Network: $W_1 \in \mathbb{R}^{d \times 4d}$, $W_2 \in \mathbb{R}^{4d \times d}$ (expansion factor 4)

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}W_1 + b_1)W_2 + b_2 \tag{24}$$

39 / 1

ViT OUTPUT: CLASSIFICATION



- Final Encoder Output:
 - After L layers: $\mathbf{z}_2^{(L)} \in \mathbb{R}^{(N+1) \times d}$
 - Extract [CLS] token representation: $\mathbf{z}_{[\text{CLS}]}^{(L)} \in \mathbb{R}^d$
 - This contains aggregated information from entire image
- Classification Head (MLP):

$$\mathbf{h} = \text{LayerNorm}(\mathbf{z}_{[\text{CLS}]}^{(L)}) \tag{25}$$

$$\text{logits} = \mathbf{h}W_{\text{head}} + b_{\text{head}} \tag{26}$$

where $W_{\text{head}} \in \mathbb{R}^{d \times K}$, K is number of classes

- Softmax for Probabilities:
$$P(y = k|\mathbf{I}) = \text{softmax}(\text{logits})_k \tag{27}$$
 - Output: Probability distribution over K classes
 - Final prediction: $\hat{y} = \arg \max_k P(y = k|\mathbf{I})$
- Training: Cross-entropy loss between predictions and true labels

40 / 1

ViT VS CNNs: TRADE-OFFS



Aspect	CNNs	ViT
Inductive Bias	Strong (locality, translation equivariance)	Weak (learns from data)
Data Efficiency	Good with small datasets	Requires large-scale pre-training
Scalability	Limited by kernel size	Scales well with model size
Global Context	Limited (small receptive field)	Excellent (self-attention)
Compute	Efficient convolutions	Quadratic in sequence length
Transfer Learning	Good	Excellent (better than CNNs)

- Practical Guidelines:
 - Use CNNs:** Small datasets (<100K images), limited compute
 - Use ViT:** Large-scale pretraining available, transfer learning
 - Hybrid:** Combine CNN stem with Transformer body

41 / 1

TABLE OF CONTENTS



- 1 Motivation and Industry Applications
- 2 Transformer Architecture
- 3 Encoder Architecture
- 4 Decoder Architecture
- 5 Vision Transformer
- 6 Transformer Variants
- 7 Using Pre-trained Models

42 / 1

WHY THREE TRANSFORMER VARIANTS?



- Different Tasks Need Different Architectures

- ▶ Not all NLP tasks require generation
- ▶ Not all tasks need bidirectional context
- ▶ Trade-offs between capability and efficiency

- Key Design Decision: Attention Pattern

Aspect	Encoder-Only	Decoder-Only	Encoder-Decoder
Attention	Bidirectional	Causal (masked)	Both
Information Flow	All tokens see all	Token sees only past	Encoder: bi, Decoder: causal
Task Type	Understanding	Generation	Transformation
Use Case	Full context needed	Sequential output	Input $\neq$ Output
Efficiency	Single pass, parallel	Simpler, scalable	Most flexible, compute-intensive

43 / 1

ENCODER-ONLY ARCHITECTURE



- Architecture:

- ▶ Only encoder stack (no decoder)
- ▶ Bidirectional self-attention: Each token attends to all tokens
- ▶ Output: Rich contextual representations for each input token

- Pretraining: Masked Language Modeling (MLM)

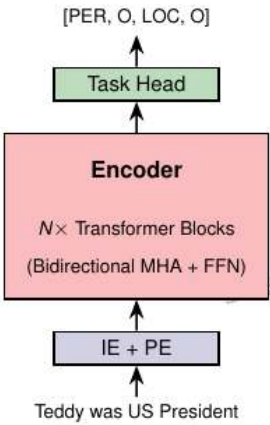
- ▶ Randomly mask 15% of input tokens
- ▶ Predict masked tokens using context from both directions
- ▶ Example: "The [MASK] sat on the mat" $\rightarrow$ predict "cat"

- Use Cases:

- ▶ Text classification: Sentiment analysis, spam detection
- ▶ Named Entity Recognition (NER): Extract names, locations
- ▶ Question Answering: Extract answer spans from context
- ▶ Sentence embeddings: Semantic similarity, clustering

44 / 1

TRANSFORMER: ENCODER-ONLY ARCHITECTURE



- Popular Models:

- BERT (2018): 110M-340M parameters
- RoBERTa (2019): Optimized BERT training
- DistilBERT (2019): 40% smaller, 60% faster

45 / 1

DECODER-ONLY ARCHITECTURE



- Architecture:

- ▶ Only decoder stack (no encoder)
- ▶ Causal/masked self-attention: Token t attends only to $\leq t$
- ▶ Output: Probability distribution over next token

- Pretraining: Autoregressive Language Modeling

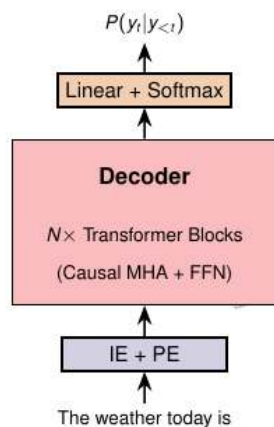
- ▶ Predict next token given previous tokens
- ▶ Training objective: $P(x_t | x_{<t})$
- ▶ Example: "The cat sat on" $\rightarrow$ predict "the"

- Use Cases:

- ▶ Text generation: Creative writing, content creation
- ▶ Conversational AI: ChatGPT, Claude, Gemini
- ▶ Code generation: GitHub Copilot, CodeLlama
- ▶ Few-shot learning: In-context learning without fine-tuning

46 / 1

TRANSFORMER: DECODER-ONLY ARCHITECTURE



Popular Models:

- GPT-4 (2023): 1.8T parameters (estimated)
- Claude 4 (2025): 200B parameters
- LLaMA 3 (2024): 8B-405B parameters (open)
- Mistral (2023): 7B-123B parameters (open)

47 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ENCODER-DECODER ARCHITECTURE



Architecture:

- ▶ Full transformer: Both encoder and decoder stacks
- ▶ Encoder: Bidirectional self-attention on input
- ▶ Decoder: Causal self-attention + cross-attention to encoder

Pretraining: Various Objectives

- ▶ T5: Text-to-text format (all tasks as generation)
- ▶ BART: Denoising autoencoder (corrupt and reconstruct)
- ▶ mT5: Multilingual text-to-text

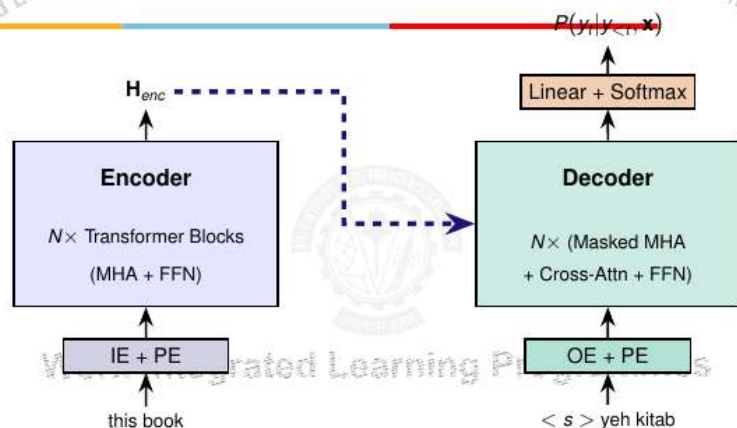
Use Cases:

- ▶ Machine translation: English → French, etc.
- ▶ Summarization: Long document → concise summary
- ▶ Question answering: Question + context → answer
- ▶ Data-to-text: Structured data → natural language

48 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ENCODER-DECODER TRANSFORMER



Popular Models:

- T5 (2019): 60M-11B parameters
- mT5 (2020): Multilingual T5 (101 languages)

49 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRAINING VS INFERENCE: EXAMPLES



Encoder-Decoder (Translation):

Training:

- ★ Input: "this book is good" → Target: "yeh kitab acchi hai"
- ★ Decoder sees all target tokens with masking
- ★ Predicts all tokens in parallel (teacher forcing)

Inference:

- ★ Step 1: ["<start>"] → "yeh"
- ★ Step 2: ["<start>", "yeh"] → "kitab" (continues...)

Encoder-Only (Named Entity Recognition):

- ▶ Input 1: "Teddy is a soft toy" → Output: [O, O, O, O, O] (O=other)
- ▶ Input 2: "Teddy was US President" → Output: [PER, O, LOC, O]
- ▶ Processes entire sentence at once (bidirectional context)

Decoder-Only (Text Completion):

- ▶ Prompt: "The weather today is"
- ▶ Generates: "sunny and warm with a gentle breeze"
- ▶ Each token generated based on all previous tokens

50 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

COMPARISON: WHEN TO USE WHAT

Aspect	Encoder-Only	Decoder-Only	Encoder-Decoder
Attention	Bidirectional	Causal	Both
Use Case	Understanding	Generation	Transformation
Tasks	Classification, NER, QA	Chat, completion	Translation, summary
Examples	Sentiment, entity tagging	Code gen, dialogue	Eng→Hindi, abstractive
Models	BERT, RoBERTa	GPT-4, Claude, LLaMA	T5, BART, mT5
Context	Full sentence	Left-to-right	Source: full, Target: causal
Speed	Fast (parallel)	Slow (sequential)	Medium

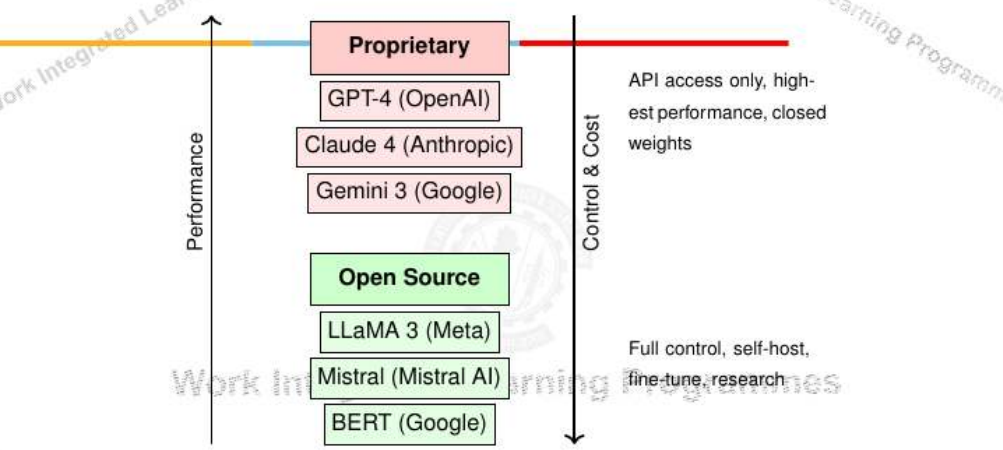
Selection Guide:

- Need to classify or extract? → Encoder-only
- Need to generate text? → Decoder-only
- Input language ≠ Output language? → Encoder-decoder

LARGE-SCALE PRETRAINING PARADIGM

- Why Pretrain?
 - ▶ Learn general language understanding from massive unlabeled data
 - ▶ Transfer knowledge to downstream tasks with limited labeled data
 - ▶ Better performance than training from scratch
- Pretraining Objectives by Architecture:
 - ▶ **Encoder-only**: Masked Language Modeling (BERT, RoBERTa)
 - ▶ **Decoder-only**: Autoregressive LM (GPT, LLaMA)
 - ▶ **Encoder-decoder**: Span corruption, denoising (T5, BART)
- Fine-tuning Approaches:
 - ▶ Full fine-tuning: Update all parameters
 - ▶ Parameter-efficient: LoRA, adapters (update <1% params)
 - ▶ Prompting: Zero-shot, few-shot (no parameter updates)

FOUNDATION MODELS LANDSCAPE



- Trade-offs:
- ▶ Proprietary: Best performance, no control, API costs
 - ▶ Open: Full control, hosting costs, requires expertise

TABLE OF CONTENTS

- 1 Motivation and Industry Applications
- 2 Transformer Architecture
- 3 Encoder Architecture
- 4 Decoder Architecture
- 5 Vision Transformer
- 6 Transformer Variants
- 7 Using Pre-trained Models



What is HuggingFace?

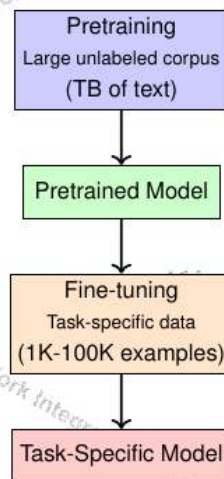
- Open platform for ML models, datasets, and applications
- 500,000+ pre-trained models available
- Easy-to-use transformers library (Python)
- Community-driven: researchers, companies, hobbyists
- Popular Models on HuggingFace:**
 - Encoder-only:** bert-base-uncased, roberta-large, distilbert-base
 - Decoder-only:** gpt2, meta-llama/Llama-3-8B, mistralai/Mistral-7B
 - Encoder-decoder:** t5-base, facebook/bart-large, google/flan-t5-large
- Benefits:** Instant access, no training needed, proven performance

55 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



Transfer Learning Paradigm:



Self-supervised: MLM, autoregressive

Supervised: Classification, NER, etc.

Why Not Train From Scratch?

- Requires massive data (billions of tokens)
- Expensive compute (thousands of GPU-hours)
- Pretrained models already learned general language patterns

56 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FINE-TUNING WORKFLOW



- Step 1: Load Pretrained Model**
- Step 2: Prepare Task-Specific Data**
 - Tokenize inputs with pretrained tokenizer
 - Create train/validation splits
 - Prepare labels in correct format
- Step 3: Fine-tune with Small Learning Rate**
 - Typical LR: 2×10^{-5} to 5×10^{-5}
 - Few epochs: 2-5 (avoid overfitting)
 - Use Adam optimizer with weight decay
- Step 4: Evaluate on Validation Set**
 - Monitor metrics: Accuracy, F1, loss
 - Early stopping if validation performance degrades

57 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FINE-TUNING EXAMPLE: SENTIMENT ANALYSIS



- Task:** Classify movie reviews as positive/negative
- Dataset:** IMDb (25K train, 25K test)
- Base Model:** BERT or RoBERTa

```

from transformers import (AutoTokenizer, Trainer,
                          AutoModelForSequenceClassification, TrainingArguments)

# Load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2
)

# Tokenize data
def tokenize_function(examples):
    return tokenizer(examples['text'], padding='max_length',
                    truncation=True, max_length=512)

tokenized_train = train_dataset.map(tokenize_function, batched=True)
tokenized_val = val_dataset.map(tokenize_function, batched=True)
  
```

58 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



```
# Training arguments
training_args = TrainingArguments(
    output_dir='./results', num_train_epochs=3,
    per_device_train_batch_size=16, per_device_eval_batch_size=64,
    learning_rate=2e-5, weight_decay=0.01,
    evaluation_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True,
)

# Trainer
trainer = Trainer(model=model, args=training_args,
    train_dataset=tokenized_train, eval_dataset=tokenized_val,
)

# Train
trainer.train()

# Evaluate
results = trainer.evaluate()
print(f"Accuracy: {results['eval_accuracy']:.4f}")
```

59 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



- **Learning Rate:**
 - ▶ Use small learning rates: 2×10^{-5} to 5×10^{-5}
- **Layer Freezing Strategies:**
 - ▶ **Full fine-tuning:** Update all parameters (best performance)
 - ▶ **Gradual unfreezing:** Start with top layers, unfreeze gradually
 - ▶ **Freeze embeddings:** Only update task-specific layers
- **Hyperparameter Selection:**
 - ▶ Batch size: 16-32 (balance memory and convergence)
 - ▶ Epochs: 2-5 (more epochs → overfitting risk)
 - ▶ Weight decay: 0.01-0.1 (regularization)
 - ▶ Gradient clipping: Max norm 1.0 (stability)
- **Data Considerations:**
 - ▶ Need at least 1,000 labeled examples for fine-tuning
 - ▶ More data (10K-100K) → better performance

60 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



- **Classification Tasks:**
 - ▶ **Accuracy:** $\frac{\text{Correct predictions}}{\text{Total predictions}}$
 - ▶ **F1 Score:** Harmonic mean of precision and recall
 - ▶ **Confusion Matrix:** Detailed breakdown of predictions
- **Named Entity Recognition (NER):**
 - ▶ **Entity-level F1:** Precision and recall on full entities
 - ▶ **Token-level accuracy:** Per-token classification accuracy
- **Question Answering:**
 - ▶ **Exact Match (EM):** Percentage of exact answer matches
 - ▶ **F1 Score:** Token overlap between prediction and ground truth
- **Text Generation:**
 - ▶ **BLEU:** Precision-based metric for translation/generation
 - ▶ **ROUGE:** Recall-based metric for summarization
 - ▶ **Perplexity:** How well model predicts text

61 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



- **Problem:** Full fine-tuning updates all parameters
 - ▶ BERT-base: 110M parameters
 - ▶ Memory and compute intensive
- **LoRA (Low-Rank Adaptation):**
 - ▶ Freeze pretrained weights
 - ▶ Add trainable low-rank matrices: $\Delta W = BA$
 - ▶ $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$ where $r \ll d$
 - ▶ Update only $< 1\%$ of parameters

$$W_{\text{new}} = W_{\text{frozen}} + \alpha \cdot BA \quad (28)$$

- **Benefits:**
 - ▶ 10-100× fewer trainable parameters
 - ▶ Lower memory footprint
 - ▶ Faster training
 - ▶ Comparable or better performance

62 / 1

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



• What We Learned:

- ▶ Transformers revolutionized AI through pure attention
- ▶ Three architectures: Encoder-only, Decoder-only, Encoder-decoder
- ▶ Foundation of all modern LLMs and multimodal models
- ▶ HuggingFace makes pretrained models easily accessible
- ▶ Fine-tuning adapts models to specific tasks efficiently

• Emerging Trends:

- ▶ Multimodal Transformers: Text + Image + Audio (GPT-4V, Gemini)
- ▶ Efficiency Improvements: Sparse attention, linear attention, LoRA
- ▶ Long Context: 100K-1M token context windows (Gemini, Claude)
- ▶ Mixture of Experts (MoE): Sparse activation for efficiency

• Practical Takeaway:

- ▶ Start with pretrained models from HuggingFace
- ▶ Fine-tune on your task-specific data
- ▶ Use parameter-efficient methods for large models